GOOGLE | 1POINT3ACRES | TOPIC 3 OF 9

Dynamic Programming

3 questions to study here, each on its own page, with the full interview answer and code

9 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

1	2
HARD	MEDIUM

How to use

- Questions are Hard first, then Medium, then Easy, newest last-asked date first.
- A question that fits several topics has its pages in only one PDF; the other topics list it as SKIP and name where it is.
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Dynamic Programming | 3 questions | tap a title to jump to it

H 1. Count Perfect Wake Numbers up to N — **2026-05-07**

M 2. Grid Path Count with Diagonal Moves and Checkpoints — **2026-09-29**

M 3. Fence Painting Minimum Operations (Gemini Team) — **2026-04-27**

SKIP: these fit this topic but have their pages in another topic (9)

H SKIP Tree Distance Sum (Re-root DP) — **topic 5: Trees and Tries**

M SKIP Bounded Knapsack Variant — **topic 8: Greedy and Math**

M SKIP Longest Non-Decreasing Subarray with One Replacement — **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Minimum-Cost Tree Disconnection — **topic 5: Trees and Tries**

M SKIP Target Expressions from Ordered Digits — **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Count Word Decompositions into Chemical Element Symbols — **topic 2: Strings, Parsing and Recursion**

M SKIP Jump Game with Profit — **topic 1: Arrays, Hashing and Two Pointers**

M SKIP Count Same-Color Squares in Matrix — **topic 6: Grids and Simulation**

M SKIP Russian Doll Envelopes — **topic 1: Arrays, Hashing and Two Pointers**

STUDY HERE: topic 3, Dynamic Programming

Also fits: Greedy and Math (listed there as SKIP, no pages). Site tags: dp, bit-manipulation, hard

HARD | 1Point3Acres | phone-screen, onsite-coding | 1 of 3

Count Perfect Wake Numbers up to N

ASKED 2026-05-07 | 166 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 187/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-05-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/perfect-wake-number-count

QUESTION

EXAMPLES

```
196   -> perfect (digits 1,9,6 all distinct, no 0, 9
    is not a valley)
23    -> perfect
12463 -> perfect (1<2, 2<4, 4>6? No, 4<6, 6>3 — 6 is
    not a valley because its right neighbor 3 is
    smaller)
1546  -> not perfect (4 is adjacent to 5 and 6, both
    strictly larger)
320   -> not perfect (contains 0)
34321 -> not perfect (digit 3 repeats)
```

APPROACH

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate." Practice bitmask-over-digits + tight-flag state machines. Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact definition: no zero digit, all digits distinct, no interior digit smaller than both neighbors. Ask for the range of N and whether to count 1..N inclusive.

BRUTE FORCE TO SAY OUT LOUD

Test every number from 1 to N with a digit check. O(N * d), fine for small N.

OPTIMAL IDEA

Digit DP over positions with state (used-digit mask, last digit, whether the digits have started falling, tight flag). Numbers with fewer digits than N are counted without a tight constraint, and zero is never allowed, so there are no leading-zero cases. Digits must rise then fall, which is exactly the no-valley rule. There is also a closed form of C(9, k) * 2^(k - 1) for k-digit numbers.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

196, 23, 12463 perfect; 1546, 320, 34321 not perfect. N = 9, N = 10, N a power of ten.

LIKELY FOLLOW-UPS

A range [A, B] (inclusion-exclusion), or different digit rules.

PITFALLS

Forgetting the zero rule. Letting a digit repeat across the tight and free branches.

HOW TO THINK ABOUT IT

Distinct digits with no zero and no valley means the digits go up and then down (they can only change direction once). That structure is both a closed form for each length and a state for a digit DP: position, which digits are used, the last digit, and whether the digits have started falling. The tight flag handles the bound N.

Basis: Digit DP, like LeetCode 1012 (Numbers With Repeated Digits) and 233. Definition from the report: no zero digit, all digits distinct, and no interior digit is a valley (strictly smaller than both neighbours).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_perfect(x):
    d = list(map(int, str(x)))
    if 0 in d or len(set(d)) < len(d):
        return False
    for i in range(1, len(d) - 1):
        if d[i] < d[i - 1] and d[i] < d[i + 1]:
            return False  # a valley
    return True
count = 0
for x in range(1, N + 1):
    count += is_perfect(x)
```

TIME	SPACE
O(N * d)	O(1)

WHY IT IS SLOW

O(N * digits): hopeless for N around 10^9. A digit DP counts by position, used digits, last digit and direction in time independent of N's size.

OPTIMAL SOLUTION · Part 1: test a single number

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: digit DP with tight flag, used-digit mask, last digit and direction

TIME	SPACE
O(d * 2^9 * 9 * 9)	O(2^9 * 9)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

FOLLOW-UP 2 How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

FOLLOW-UP 3 What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) * 2^(k-1). Only the 'at most N' bound needs the DP.

FOLLOW-UP 4 Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, k) ways); the largest must be the peak, and each of the other k - 1 digits goes either left or right of it in sorted order, so 2^(k-1) arrangements: C(9, k) * 2^(k-1). Only the bound N needs the DP.

FOLLOW-UP 5 Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from functools import lru_cache
from math import comb


# BASE QUESTION (optimal): test a single number
def is_perfect(n):
    # distinct digits, no zero, and no valley: no digit is
    # smaller than both of its neighbours
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1])
               for i in range(1, len(d) - 1))


# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
def with_k_digits(k):
    # choose the k digits; the largest is the peak and
    # each
    # other digit goes left or right of it
    return comb(9, k) * 2 ** (k - 1) if k >= 1 else 0


# FOLLOW-UP 2: How do you count the numbers in a range [A,
# B]?
# PART 2: count the perfect numbers up to N
def count_up_to(n):
    if n < 1:
        return 0
    digits = list(map(int, str(n)))
    size = len(digits)
    # all shorter numbers: closed form
    total = sum(with_k_digits(k) for k in range(1, size))

    @lru_cache(None)
    def go(pos, mask, last, falling, tight):
        # count completions of a number of full length
        if pos == size:
            return 1
        out = 0
        limit = digits[pos] if tight else 9
        for d in range(1, limit + 1):
            if mask >> d & 1:
                continue
            fall = falling
            if pos:
                if d > last and falling:
                    # going up after going down: a valley
                    continue
                if d < last:
                    fall = True
            out += go(pos + 1, mask | 1 << d, d, fall,
                      tight and d == digits[pos])
        return out

    # numbers with the same length as n, not above n
    return total + go(0, 0, 0, False, True)


# FOLLOW-UP 5: Count perfect numbers in a range [A, B].
def count_between(a, b):
    # inclusion-exclusion over the upper bounds
    return count_up_to(b) - count_up_to(a - 1)


if __name__ == "__main__":
    for ok in (196, 23, 12463):
        assert is_perfect(ok)
    for bad in (1546, 320, 34321):
        assert not is_perfect(bad)
    # the DP agrees with checking every number
    for n in (0, 9, 10, 99, 100, 999, 5000, 20000):
        every = range(1, n + 1)
        brute = sum(is_perfect(i) for i in every)
        assert count_up_to(n) == brute
```

```
    assert count_between(10, 99) == (
        count_up_to(99) - count_up_to(9))
    # 1-digit and 2-digit counts from the closed form
    assert with_k_digits(1) == 9
    assert with_k_digits(2) == 72
```

NOTES FROM CANDIDATES

Part 1 is a linear scan; Part 2 is digit DP with state (position, mask of used digits, last digit, is_tight against n). Track whether the previous step went up or down to detect a future valley.
Multiple candidates report not finishing Part 2 in the 45-minute window. The interviewer is generally patient and dispenses hints ("think DP" / "think digit DP") when you stall.
Watch out: the "no valley" rule is about interior digits, so the first and last digit are never themselves the valley; they can still create one for their neighbor.
The full prompt is short and recent — expect the follow-up to vary (e.g. count exactly-K-digit Perfect Wake Numbers).

PREPARATION

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate."
Practice bitmask-over-digits + tight-flag state machines.
Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

STUDY HERE: topic 3, Dynamic Programming

Also fits: Grids and Simulation (listed there as SKIP, no pages). Site tags: dp, grid, medium

MEDIUM | 1Point3Acres | phone-screen | 2 of 3

Grid Path Count with Diagonal Moves and Checkpoints

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 48.9** | **30 DAYS 33** | **PRIORITY 35/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #62 Unique Paths

LeetCode match: LC 62 Unique Paths (related)

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/grid-path-count-three-moves-checkpoints

QUESTION

Count paths in an n x m matrix from (n-1, 0) to (n-1, m-1) moving up-right, right, or down-right. Follow-ups: count paths visiting all checkpoints; then checkpoints must be visited in order, with the observation that each move changes column, and optimize space.

Matrix of size n x m. Start at (n-1, 0), end at (n-1, m-1). Allowed moves from (r, c): up-right (r-1, c+1), right (r, c+1), down-right (r+1, c+1); stay inside the matrix.
Return the total number of distinct paths from start to end.
Follow-up 1: the matrix contains checkpoints; count paths that visit all checkpoints.
Follow-up 2: the checkpoints must be visited in order. Since every move changes the column, each column can contain at most one visited checkpoint. Count the paths and optimize space.

APPROACH

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

HOW TO THINK ABOUT IT

Every move goes exactly one column to the right, so a path visits each column exactly once and the state is just 'which row am I in at this column'. That is a DP over columns with a single rolling array. A checkpoint (r, c) simply says the path must be in row r at column c, so you zero every other row of that column.

Basis: LeetCode 62, Unique Paths, with up-right / right / down-right moves, and checkpoints the path must visit.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def count(r, c, seen):
    if c == m - 1:
        done = r == n - 1 and checkpoints <= seen
        return 1 if done else 0
    total = 0
    for dr in (-1, 0, 1):
        nr = r + dr
        if 0 <= nr < n:
            nxt = seen | {(nr, c + 1)}
            total += count(nr, c + 1, nxt)
    return total
answer = count(n - 1, 0, {(n - 1, 0)})
```

TIME	SPACE
O(3^m)	O(m)

WHY IT IS SLOW

Up to 3^m paths. Every move advances one column, so a dp over (column, row) with a rolling array counts them in O(n * m).

OPTIMAL SOLUTION · Column DP with a rolling array and checkpoint filters

TIME	SPACE
O(n * m)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two checkpoints share a column.

No path can visit both, because a path is in exactly one row per column, so the answer is 0 (the code returns it before the DP). Checkpoints at the start or end cell are fine: they only filter rows of the first or last column.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def count_paths_table(n, m):
    # full table: ways[c][r] paths reach row r, column c
    ways = [[0] * n for _ in range(m)]
```

```
    ways[0][n - 1] = 1
    for c in range(1, m):
        for r in range(n):
            # arrive from up-right, right or down-right
            for pr in (r - 1, r, r + 1):
                if 0 <= pr < n:
                    ways[c][r] += ways[c - 1][pr]
    return ways[m - 1][n - 1]


def step(ways):
    # next column from the current column only
    n = len(ways)
    return [(ways[r - 1] if r else 0) + ways[r]
            + (ways[r + 1] if r + 1 < n else 0)
            for r in range(n)]


def keep_row(ways, row):
    # a checkpoint forces the path through one row
    return [x if r == row else 0
            for r, x in enumerate(ways)]


# FOLLOW-UP 1: Two checkpoints share a column.
def checkpoint_rows(checkpoints, ordered):
    # column -> required row; None if impossible
    rows = {}
    for r, c in checkpoints:
        # two different checkpoints in one column: no path
        if rows.setdefault(c, r) != r:
            return None
    if len(rows) != len(checkpoints):
        return None
    cols = [c for _, c in checkpoints]
    # ordered: they must be listed left to right
    if ordered and cols != sorted(cols):
        return None
    return rows


# BASE QUESTION (optimal): Column DP with a rolling array
```

```
# and checkpoint filters
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    # one rolling column is enough: O(n) space
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]


if __name__ == "__main__":
    assert count_paths(3, 3) == 2
    assert count_paths_table(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths_table(4, 6) == count_paths(4, 6)
    # force the middle column through row 1
    assert count_paths(3, 3, [(1, 1)]) == 1
    # two checkpoints in one column: impossible
    assert count_paths(3, 3, [(0, 1), (1, 1)]) == 0
    # ordered checkpoints must go left to right
    assert count_paths(3, 4, [(1, 1), (1, 2)], True) == 1
    assert count_paths(3, 4, [(1, 2), (1, 1)], True) == 0
```

NOTES FROM CANDIDATES

Asked in an L4 phone interview.
Every move advances exactly one column, so the path visits each column exactly once — this is the key structural fact the follow-ups build on.
Clarify for follow-up 1 what happens if two checkpoints share a column (no path can visit both) and whether checkpoints are given as a list or marked in the grid.

PREPARATION

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

STUDY HERE: topic 3, Dynamic Programming

Also fits: Strings, Parsing and Recursion (listed there as SKIP, no pages). Site tags: dp, recursion, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 3 of 3

Fence Painting Minimum Operations (Gemini Team)

ASKED 2026-04-27 | 176 DAYS AGO | LAST 6 MONTHS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 200/228

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/fence-painting-min-ops

QUESTION

EXAMPLES

```
n = 5, a = [2, 2, 1, 2, 1]
Optimal:
  1) Horizontal stroke at h=1 across [1..5]
      (covers row 1 across whole fence)
  2) Horizontal stroke at h=2 across [1..2]
  3) Vertical stroke at plank 4 (height 2 still
      missing row 2)
Optimal answer: 3.
```

APPROACH

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether vertical strokes paint one plank fully, whether horizontal strokes must cover consecutive planks at one height, and the sizes involved.

BRUTE FORCE TO SAY OUT LOUD

Try all combinations of strokes. Exponential.

OPTIMAL IDEA

Divide and conquer on the minimum height: for a range with base height h, painting every plank vertically costs the number of planks; otherwise paint one horizontal band up to the minimum height m (cost 1) and recurse on the segments above m, split where planks are exactly m. The answer is the minimum of the two. O(n^2) worst case, O(n log n) with a sparse table or a min-stack for range minimum queries.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The example giving 3. All equal heights (1). Strictly increasing heights. A single plank.

LIKELY FOLLOW-UPS

A brush of width W, or return the strokes.

PITFALLS

Forgetting the vertical option, so a tall single plank is overcounted. Off-by-one on the band height.

HOW TO THINK ABOUT IT

The shortest plank in a range decides the shape: a horizontal band along the bottom, up to that plank's height, is free of regret because every plank in the range needs it. That splits the range at the shortest planks, so recurse on the pieces above the band. The other choice for a range is to paint every plank vertically. Take whichever is cheaper.

Basis: Codeforces 448C, Painting Fence. Strokes: a vertical stroke paints one whole plank; a horizontal stroke paints one row across consecutive planks that all reach that height.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def paint(l, r, base):
    if l > r:
        return 0
    m = min(height[l:r + 1])
    vertical = r - l + 1
    horizontal = m - base
    for part in split_at_height(l, r, m):
        horizontal += paint(part.l, part.r, m)
    return min(vertical, horizontal)
```

TIME	SPACE
O(n^2)	O(n)
worst case	recursion

WHY IT IS SLOW

Finding the minimum by scanning makes a sorted or constant fence quadratic, O(n^2). A sparse table for range-minimum gives O(n log n).

OPTIMAL SOLUTION · Same recursion with a sparse table for range minimum

TIME	SPACE
O(n log n)	O(n log n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The horizontal brush is only W planks wide.

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (m - base) * ceil(len / W). The vertical option and the splitting are unchanged.

FOLLOW-UP 2 Return the strokes, not just the count.

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

FOLLOW-UP 3 Is it the same as largest rectangle in a histogram?

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. **Cost:** the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def sparse_table(a):
    # table[j][i] = index of the smallest plank in a[i : i
    # + 2^j]
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        prev, row = table[-1], []
        for i in range(len(a) - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1
    return table


# FOLLOW-UP 2: Return the strokes, not just the count.
def paint_strokes(a, width=None):
    # returns (number of strokes, the list of strokes)
    n = len(a)
    if n == 0:
        return 0, []
    table = sparse_table(a)

    def argmin(l, r):
        # smallest plank in a[l..r] in O(1)
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        # paint planks l..r whose bottom `base` rows are
        # done
        if l > r:
            return 0, []
        m = argmin(l, r)
        size = r - l + 1
        # a band of rows over the whole range needs one
        # stroke per row (or per `width` planks)
        per_row = 1
        if width is not None:
            per_row = -(-size // width)
        band = (a[m] - base) * per_row
        lc, ls = solve(l, m - 1, a[m])
        rc, rs = solve(m + 1, r, a[m])
        horizontal = band + lc + rc
        # or paint every plank by itself, top to bottom
        if size <= horizontal:
            planks = range(l, r + 1)
            return size, [("vertical", i) for i in planks]
        rows = [("horizontal", h, l, r)
                for h in range(base + 1, a[m] + 1)]
        return horizontal, rows + ls + rs

    return solve(0, n - 1, 0)


# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
def paint(a, width=None):
    # FOLLOW-UP 1: width= limits the horizontal brush
    return paint_strokes(a, width)[0]


if __name__ == "__main__":
    # the example: 3 strokes
    assert paint([2, 2, 1, 2, 1]) == 3
    assert paint([]) == 0
    assert paint([3]) == 1
    # a flat fence is one stroke per row
    assert paint([4, 4, 4, 4]) == 4
    # tall thin planks are cheaper to paint upright
    assert paint([5, 0, 5]) == 2
    count, strokes = paint_strokes([2, 2, 1, 2, 1])
    assert count == len(strokes) == 3
    # a narrow brush needs several strokes per row
    assert paint([2, 2, 2, 2], width=2) == 4
    assert paint([2, 2, 2, 2], width=4) == 2
```

NOTES FROM CANDIDATES

Classic recursive divide-and-conquer: for range [l..r] with base height h:
Option A: paint every plank vertically -> cost r - l + 1.
Option B: paint a horizontal band of height min - h, then recurse on the sub-ranges above the band split by planks at height min. Take the minimum.
Equivalent to Cartesian-tree DP; runs in O(N log N) with min-stack or sparse table, or O(N²) with the recursive form.
Verbalize the recursion's correctness: the band optimally covers all planks >= min in the range, then each "taller column block" is solved independently.

PREPARATION

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.